

Simulare buchi neri

Francesco Sermi

29 novembre 2025

Lo scopo di questo breve (o almeno spero) documento è quello di riportare in forma abbastanza elegante tutti i risultati matematici e spiegazioni necessarie per simulare un buco nero in Python (o in C++ se vedo che non è possibile).

1 Requisiti matematicosi

Il punto di partenza è sicuramente la metrica di Schwarzschild

$$ds^2 = -\left(1 - \frac{2GM}{r}\right)dt^2 + \left(1 - \frac{2GM}{r}\right)^{-1}dr^2 + r^2(d\theta^2 + \sin^2\theta d\phi^2),$$

da cui segue che il nostro tensore metrico

$$g_{\mu\nu} = \begin{pmatrix} -(1 - \frac{2GM}{r}) & 0 & 0 & 0 \\ 0 & (1 - \frac{2GM}{r})^{-1} & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin^2\theta \end{pmatrix}$$

Visto che tengo alla mia sanità mentale, vorrei, innanzitutto, introdurre la notazione di Einstein che mi permetterà di snellire particolarmente le formule: se abbiamo per esempio due vettori, v e w , le cui componenti verranno indicate con v^μ e w^μ rispettivamente, avremo che scrivere

$$v^\mu w_\mu,$$

dove abbiamo un indice ripetuto indica che abbiamo una sommatoria non esplicita su tale indice, dunque

$$v^\mu w_\mu = \sum_{\mu=0}^3 v^\mu w_\mu = v^0 w_0 + v^1 w_1 + v^2 w_2 + v^3 w_3.$$

In accordo con la letteratura, diremo che un oggetto con un indice in alto trasforma in maniera *controvariante* altrimenti trasforma in maniera *covariante*:

Definizione 1.1 (trasformazioni covarianti e controvarianti). Diremo che un oggetto v che da un sistema di riferimento ad un altro (con jacobiano $\xi^\mu_\nu = \frac{\partial(x')^\mu}{\partial x^\nu}$) trasforma in maniera controvariante se

$$(v')^\mu = \xi^\mu_\nu v^\nu,$$

e gli posizioneremo l'indice in alto. Altrimenti, diremo che trasforma in maniera covariante se

$$(v')_\mu = \xi^\nu_\mu v_\nu,$$

e gli posizioneremo l'indice in basso.

Chiaramente esistono degli oggetti con più indici: i tensori, che possono avere p indici in alto e q indici in basso, che sono delle applicazioni che mappano p vettori e q covettori (funzionali) in un numero¹,

$$\Lambda_{\beta_1, \dots, \beta_q}^{\alpha_1, \dots, \alpha_p},$$

che trasforma come segue

$$(\Lambda')_{\nu_1, \dots, \nu_q}^{\mu_1, \dots, \mu_p} = \xi_{\alpha_1}^{\mu_1} \xi_{\alpha_2}^{\mu_2} \dots \xi_{\alpha_p}^{\mu_p} \xi_{\nu_1}^{\beta_1} \xi_{\nu_2}^{\beta_2} \dots \xi_{\nu_q}^{\beta_q} \Lambda_{\beta_1, \dots, \beta_q}^{\alpha_1, \dots, \alpha_p}.$$

Abbiamo che i simboli di Christoffel in questa metrica sono dati da

$$\Gamma_{\mu\nu}^{\sigma} = \frac{1}{2} g^{\sigma\rho} (\partial_{\mu} g_{\nu\rho} + \partial_{\nu} g_{\rho\mu} - \partial_{\rho} g_{\mu\nu}), \quad \partial_{\mu} = \frac{\partial}{\partial x^{\mu}},$$

Facendo il conto² (facendolo fare a Wolfram Mathematica) si vede che

$$\Gamma_{ij}^{\phi} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{r} \\ 0 & 0 & 0 & \cot \theta \\ 0 & \frac{1}{r} & \cot \theta & 0 \end{pmatrix}, \quad \Gamma_{ij}^{\theta} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{r} & 0 \\ 0 & \frac{1}{r} & 0 & 0 \\ 0 & 0 & 0 & -\cos \theta \sin \theta \end{pmatrix},$$

$$\Gamma_{ij}^r = \begin{pmatrix} 0 & \frac{GM}{r^2} (1 - \frac{2GM}{r})^{-1} & 0 & 0 \\ \frac{GM}{r^2} (1 - \frac{2GM}{r})^{-1} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Definizione 1.2 (metrica singolare). Diremo che una metrica è singolare se $\exists x \in M$ con M varietà riemanniana tale che $g_{\mu\nu}$ non è definito.

Vediamo che la metrica di Schwarzschild non è ben definita se $r = 2GM$ e per $r = 0$: la quantità $r_s = 2GM$ verrà indicata con *raggio di Schwarzschild*. Da Geometria differenziale sappiamo che le geodetiche³ rispettano la seguente equazione:

$$\frac{\partial^2 x^{\mu}}{\partial t^2} + \frac{\partial x^i}{\partial t} \frac{\partial x^j}{\partial t} \Gamma_{ij}^k = 0. \quad (1)$$

Quindi, il nostro scopo, sarà quello di generare un oggetto molto massivo (er buco nero) e andare ad integrare numericamente queste equazioni per ricostruire il percorso della luce: chiarissimo, no? Detto questo, veniamo alla scelta dell'integratore: visto che ancora non so niente perché ancora non ho ancora "messo le mani in pasta" (ovvero non so ancora quanto sia brutta effettivamente l'equazione, che dev'essere ancora discretizzata), effettuerò un bellissimo *educated guess* e userò l'integratore Runge-Kutta. Il primo step che dobbiamo impiegare è quello di costruire una funzione che ci calcola il tensore metrico per ogni punto considerato

```
def schwarzschild_metric(position){
    rs = 1
    r = position[1]
    theta = position[2]
    g = np.zeros(shape=[4, 4])
```

¹si possono dare anche altre definizioni (su varietà per esempio), ma ci vogliamo tenere sul semplice.

²si potrebbe anche pensare di fare noi stessi il conto, ma voglio alleggerire il carico di lavoro al mio povero PC, quindi si fa analiticamente il conto

³presi due punti p, q distinti su una varietà, esiste un'unica curva (che chiamiamo *geodetica*) di lunghezza minima che collega i due punti.

```

g[0][0] = -(1 - rs / r)
g[1][1] = 1 / (1 - rs / r)
g[2][2] = r * r
g[3][3] = r * r * np.sin(theta) * np.cos(theta)
return g
}

```

Successivamente bisogna calcolare le tetradi: localmente, lo spazio curvo deve assomigliare ad uno spazio euclideo, quindi bisogna calcolare i vettori di base dello spazio euclideo "curvati" dalla curvatura dello spazio-tempo: nel caso della metrica di Schwarzschild, abbiamo che questi valori sono già noti in letteratura e risultano essere pari a

$$e_0 = \left(\frac{1}{\sqrt{1 - r_s/r}}, 0, 0, 0 \right) \quad (2)$$

$$e_1 = \left(0, \frac{1}{\sqrt{1 - r_s/r}}, 0, 0 \right) \quad (3)$$

$$e_2 = \left(0, 0, \frac{1}{r}, 0 \right) \quad (4)$$

$$e_3 = \left(0, 0, 0, \frac{1}{r \sin \theta} \right). \quad (5)$$

```

def schwarzschild_tetrad(position):
    rs = 1
    r = position[1]
    theta = position[2]
    e0 = np.array([1/(np.sqrt(1-(rs/r))), 0, 0, 0])
    e1 = np.array([0, 1/np.sqrt(1-rs/r), 0, 0])
    e2 = np.array([0, 0, 1/r, 0])
    e3 = np.array([0, 0, 0, 1/(r * np.sin(theta))])
    return np.array([e0, e1, e2, e3])

```

Ora, noi vogliamo vedere tutto questo da una bella camera, dunque dobbiamo convertire i nostri dati proiettandoli su uno schermo: per fare questo si fa un pochino di trigonometria (che vi risparmio, perché geometria euclidea mi fa schifo, bensì la fucking geometria non euclidea mi garba un mondo): si converte il FOV in radianti per poi calcolare la distanza massima rispetto al piano di proiezione (la camera) sapendo che

$$d = \frac{l}{2 \tan \frac{\text{fov}}{2}},$$

per quanto riguarda le posizioni (x, y) , abbiamo che se posizioniamo il nostro *Super massive Black Hole* nell'origine e prendiamo il sistema di riferimento da cui prendere le misure lì allora

$$x = s_x - \frac{l_x}{2},$$

$$y = s_y - \frac{l_y}{2}.$$

Una volta che ci siamo calcolati la direzione del vettore, andiamo a normalizzarlo.